Practical No: 6

a. **Aim: Create a simple website for a College Information Portal and convert it into a Progressive Web Application.**

Step 1: Create a new folder

Create a new folder anywhere on your computer and name it, for example, `CollegePWA`.

Step 2: Create these 4 files

You need to create a total of 4 files inside that folder (you can use Notepad, Visual Studio Code, or any code editor):

- **index.html (This will be the website's main page—paste the code provided earlier here).**
- **style.css (This is for the design—paste the CSS code from earlier here).**
- **manifest.json (This contains the app's details—paste the JSON code from earlier here).**
- **sw.js (This is the Service Worker that enables offline functionality—paste the JavaScript code from earlier here).**

index.html (Main Page)

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>College Information Portal</title>
    <link rel="stylesheet" href="style.css">
    <!-- PWA Manifest Link -->
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#007bff">
</head>
<body>
    <header>
        <h1>CampusConnect</h1>
        <p>Your Ultimate College Information Portal</p>
    </header>

    <main class="container">
        <section class="card">
            <h2>About Our College</h2>
            <p>Welcome to our institution, dedicated to excellence in technical education, research, and holistic student development.</p>
        </section>

        <section class="card">
            <h2>Courses Offered</h2>
            <ul>
```

```
            <li>Bachelor of Computer Applications (BCA)</li>
            <li>Master of Science in Computer Science (M.Sc. CS)</li>
            <li>Bachelor of Technology (B.Tech)</li>
        </ul>
    </section>

    <section class="card">
        <h2>Quick Announcements</h2>
        <p> Semester examination forms are now live. Last date to submit is next Friday.</p>
    </section>
</main>

<footer>
    <p>&copy; 2026 College Information Portal. All rights reserved.</p>
</footer>

<!-- Register Service Worker for PWA -->
<script>
    if ('serviceWorker' in navigator) {
        window.addEventListener('load', () => {
            navigator.serviceWorker.register('sw.js')
                .then(reg => console.log('Service Worker registered!', reg))
                .catch(err => console.log('Service Worker registration failed:', err));
        });
```

```
        }
    </script>
</body>
</html>
```

style.css (Styling)

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f7f6;
    color: #333;
}

header {
    background-color: #007bff;
    color: white;
    text-align: center;
    padding: 2rem 1rem;
}

header h1 {
    margin: 0;
    font-size: 2.2rem;
```

```
}

.container {
    padding: 20px;
    max-width: 800px;
    margin: auto;
}

.card {
    background: white;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

.card h2 {
    color: #007bff;
    margin-top: 0;
}

footer {
    text-align: center;
    padding: 15px;
    background-color: #333;
```

```
  color: white;
  position: relative;
  bottom: 0;
  width: 100%;
}
```

manifest.json (PWA Manifest)

```
{
 "name": "College Info Portal",
 "short_name": "CollegePortal",
 "start_url": "index.html",
 "display": "standalone",
 "background_color": "#f4f7f6",
 "theme_color": "#007bff",
 "icons": [
  {
   "src": "images/icon-192.png",
   "sizes": "192x192",
   "type": "image/png"
  },
  {
   "src": "images/icon-512.png",
   "sizes": "512x512",
   "type": "image/png"
```

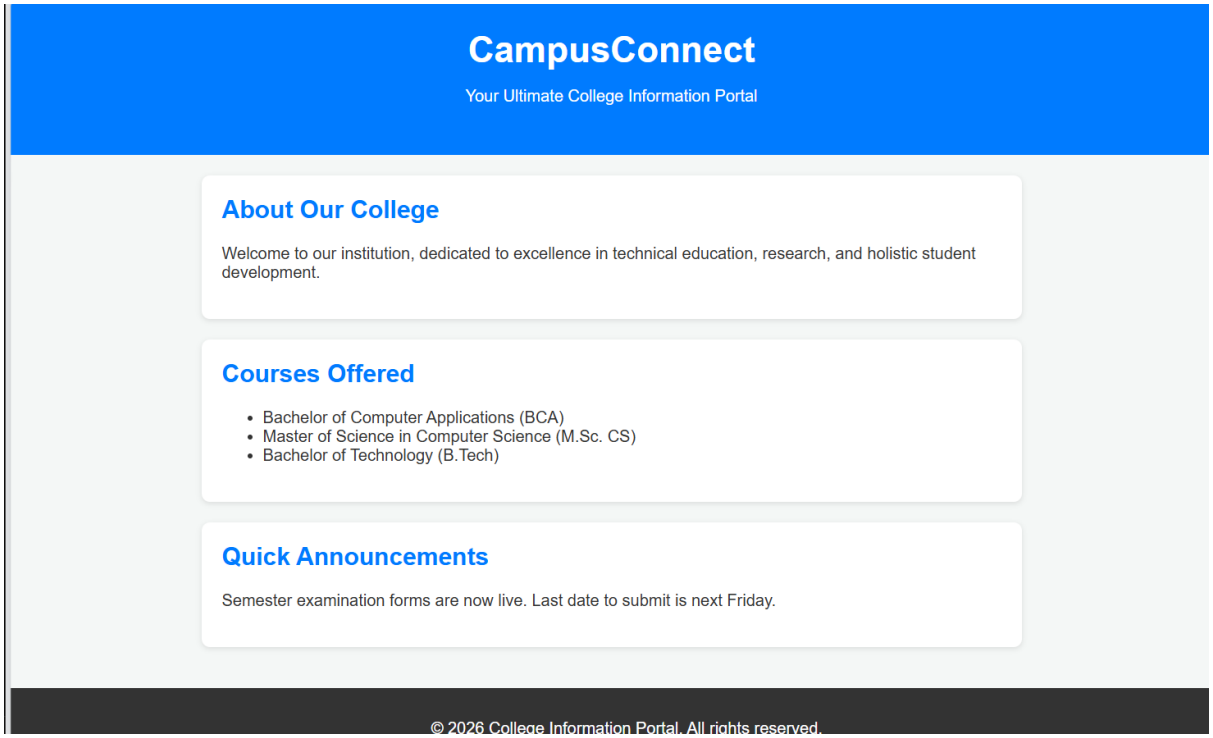
```
    }
  ]
}
```

sw.js (Service Worker for Offline Caching)

```
self.addEventListener('install', (e) => {
    e.waitUntil(
        caches.open('college-pwa-v1').then((cache) => {
            return cache.addAll([
                'index.html',
                'style.css',
                'manifest.json'
            ]);
        })
    );
});

self.addEventListener('fetch', (e) => {
    e.respondWith(
        caches.match(e.request).then((response) => {
            return response || fetch(e.request);
        })
    );
});
```

Output:



b. Create a PWA that displays a custom offline page when the user is not connected to the internet.

Offline.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Offline - College Information Portal</title>
  <style>
```

```
      body {
        font-family: Arial, sans-serif;
        text-align: center;
        padding: 50px;
        background-color: #f4f7f6;
        color: #333;
      }
      .offline-box {
        background: white;
        padding: 40px;
        border-radius: 8px;
        box-shadow: 0 2px 5px rgba(0,0,0,0.1);
        max-width: 400px;
        margin: auto;
      }
      h1 {
        color: #d9534f;
      }
      p {
        color: #666;
      }
    </style>
</head>
<body>
    <div class="offline-box">
```

```
    <h1>You are Offline</h1>
    <p>Oops! Looks like you've lost your internet connection. Please check your network to view live updates.</p>
  </div>
</body>
</html>
```

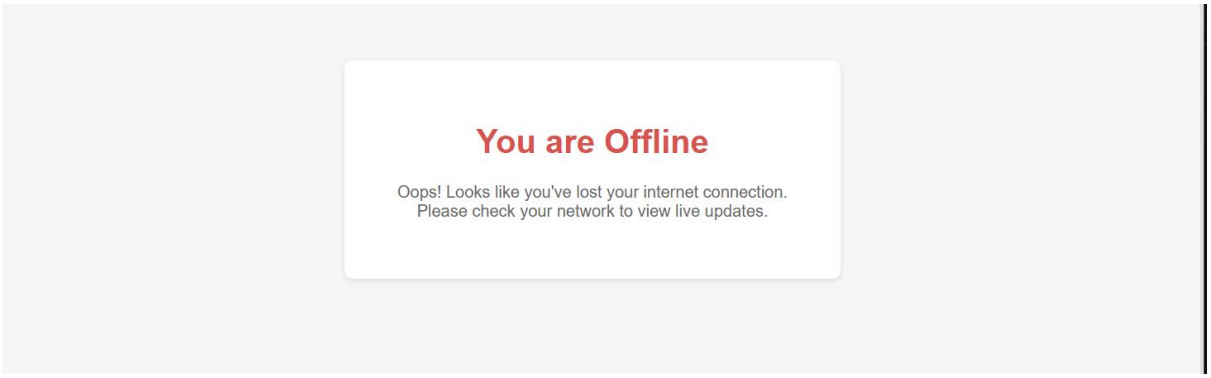
Sw.js

```
const CACHE_NAME = 'college-pwa-v2';
const assetsToCache = [
  'index.html',
  'style.css',
  'manifest.json',
  'offline.html'
];

// Install Event: Cache all essential files including offline page
self.addEventListener('install', (e) => {
  e.waitUntil(
    caches.open(CACHE_NAME).then((cache) => {
      return cache.addAll(assetsToCache);
    })
  );
});
```

```
// Fetch Event: Serve offline page if network request fails
self.addEventListener('fetch', (e) => {
   e.respondWith(
      caches.match(e.request).then((response) => {
         return response || fetch(e.request).catch(() => {
            return caches.match('offline.html');
         });
      })
   );
});
```

Output:



Practical no: 7

a. **Aim: Develop a PWA that works in offline mode using Service Worker.**

index.html (Main College Portal Page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>College Information Portal</title>
  <link rel="stylesheet" href="style.css">
  <link rel="manifest" href="manifest.json">
</head>
<body>
  <div class="container">
    <h1>CampusConnect</h1>
    <p>Your Ultimate College Information Portal</p>

    <div class="card">
      <h2>About Our College</h2>
      <p>Welcome to our institution, dedicated to excellence in
technical education, research, and holistic student development.</p>
    </div>

    <div class="card">
      <h2>Courses Offered</h2>
      <ul>
        <li>Bachelor of Computer Applications (BCA)</li>
        <li>Master of Science in Computer Science (M.Sc. CS)</li>
        <li>Bachelor of Technology (B.Tech)</li>
      </ul>
    </div>
  </div>

  <!-- Service Worker Registration Script -->
  <script>
    if ('serviceWorker' in navigator) {
      window.addEventListener('load', () => {
```

```
            navigator.serviceWorker.register('sw.js')
                .then((reg) => console.log('Service Worker registered
successfully!', reg))
                .catch((err) => console.log('Service Worker registration
failed:', err));
        });
    }
  </script>
</body>
</html>
```

offline.html (Custom Offline Page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Offline - College Information Portal</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f7f6;
      color: #333;
      text-align: center;
      padding: 80px 20px;
    }
    .offline-card {
      background: white;
      max-width: 450px;
      margin: auto;
      padding: 40px;
      border-radius: 8px;
```

```
        box-shadow: 0 4px 10px rgba(0,0,0,0.1);
      }
      h2 {
        color: #d9534f;
        margin-bottom: 10px;
      }
      p {
        color: #666;
        line-height: 1.5;
      }
    </style>
</head>
<body>
    <div class="offline-card">
      <h2>You are Offline</h2>
      <p>Oops! Looks like you've lost your internet connection. Please
check your network to view live college updates.</p>
    </div>
</body>
</html>
```

style.css (Styling sheet)

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f7f6;
    color: #333;
    margin: 0;
    padding: 20px;
}

.container {
    max-width: 800px;
    margin: 30px auto;
    background: white;
```

```
    padding: 40px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

h1 {
    color: #007acc;
    margin-bottom: 5px;
}

p {
    color: #555;
    line-height: 1.6;
}

.card {
    background: #fdfdfd;
    border-left: 4px solid #007acc;
    padding: 20px;
    margin: 20px 0;
    border-radius: 4px;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.05);
}

.card h2 {
    color: #333;
    margin-top: 0;
    margin-bottom: 10px;
}

ul {
    padding-left: 20px;
    color: #444;
}
```

```
li {
  margin-bottom: 8px;
}
```

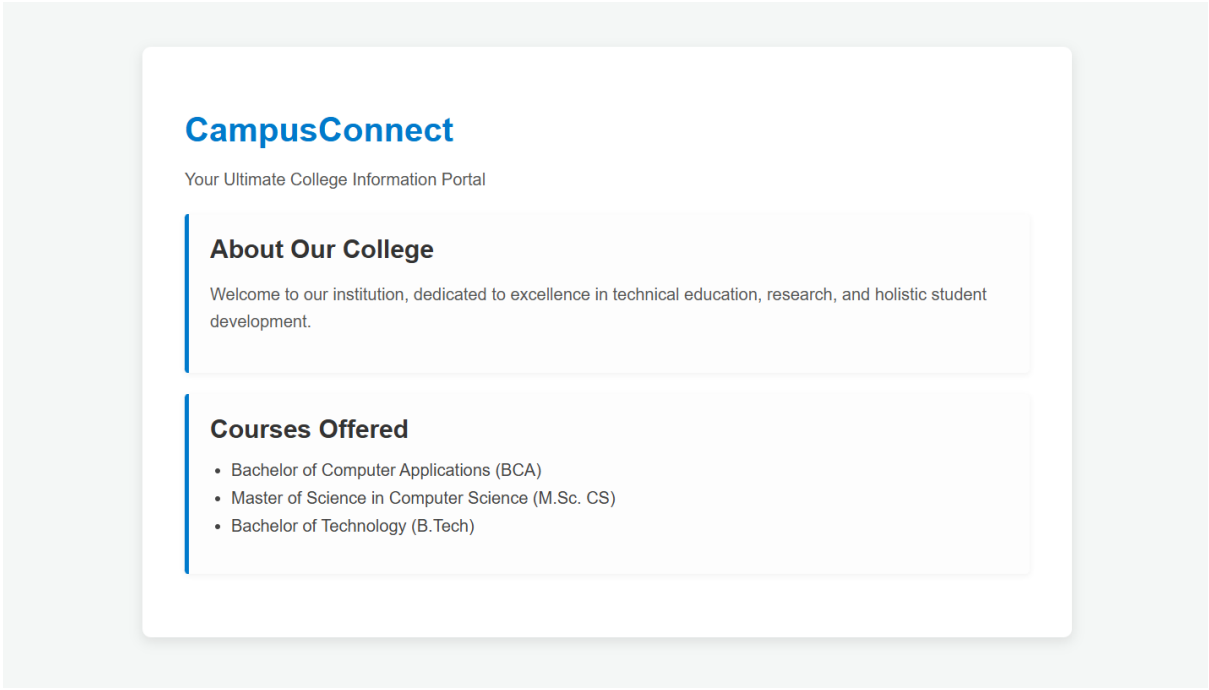
sw.js (Service Worker for managing offline cache and fallback)

```
const CACHE_NAME = 'college-offline-v1';
const ASSETS_TO_CACHE = [
  'index.html',
  'offline.html',
  'style.css'
];

// Install Event: Cache essential files
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => {
      return cache.addAll(ASSETS_TO_CACHE);
    })
  );
});

// Fetch Event: Serve custom offline page if network fails
self.addEventListener('fetch', (event) => {
  event.respondWith(
    fetch(event.request).catch(() => {
      return caches.match('offline.html');
    })
  );
});
```

Output:



b. Create a Student Record Management PWA.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Record Management PWA</title>
  <link rel="stylesheet" href="style.css">
  <link rel="manifest" href="manifest.json">
</head>
```

```
<body>
  <div class="container">
    <h1>Student Record Management</h1>

    <!-- Form to Add / Update Student -->
    <div class="form-card">
      <h3 id="formTitle">Add New Student</h3>
      <form id="studentForm">
        <input type="hidden" id="editIndex" value="">
        <input type="text" id="studentName" placeholder="Student Name" required>
        <input type="text" id="rollNo" placeholder="Roll Number" required>
        <input type="text" id="course" placeholder="Course (e.g., M.Sc. CS)" required>
        <div class="btn-group">
          <button type="submit" id="submitBtn">Add Record</button>
          <button type="button" id="cancelBtn" class="cancel-btn" style="display:none;" onclick="resetForm()">Cancel</button>
        </div>
      </form>
    </div>

    <!-- Table to Display Records -->
    <div class="table-card">
      <h3>Student Records List</h3>
```

```
        <table>
          <thead>
            <tr>
              <th>Name</th>
              <th>Roll No</th>
              <th>Course</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody id="studentTableBody">
            <!-- Records will be dynamically inserted here -->
          </tbody>
        </table>
      </div>
    </div>

    <script src="app.js"></script>
    <script>
      if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('sw.js')
          .then(() => console.log('Service Worker Registered'));
      }
    </script>
  </body>
</html>
```

style.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f7f6;
    margin: 0;
    padding: 20px;
}

.container {
    max-width: 750px;
    margin: auto;
    background: white;
    padding: 30px;
    border-radius: 8px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
}

h1 {
    color: #007acc;
    text-align: center;
    margin-bottom: 25px;
}

.form-card, .table-card {
```

```
    background: #fafafa;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 6px;
    border: 1px solid #ddd;
}

form {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

input {
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 4px;
    font-size: 14px;
}

.btn-group {
    display: flex;
    gap: 10px;
}
```

```
button {
    padding: 10px;
    background-color: #007acc;
    color: white;
    border: none;
    border-radius: 4px;
    font-size: 15px;
    cursor: pointer;
    flex: 1;
}

button:hover {
    background-color: #005999;
}

.cancel-btn {
    background-color: #6c757d;
}

.cancel-btn:hover {
    background-color: #5a6268;
}

table {
    width: 100%;
```

```
    border-collapse: collapse;
    margin-top: 10px;
}

th, td {
    padding: 10px;
    border: 1px solid #ddd;
    text-align: left;
    font-size: 14px;
}

th {
    background-color: #007acc;
    color: white;
}

.action-btns {
    display: flex;
    gap: 5px;
}

.edit-btn {
    background-color: #f0ad4e;
    color: white;
    border: none;
```

```
    padding: 5px 8px;
    border-radius: 3px;
    cursor: pointer;
}

.edit-btn:hover {
    background-color: #ec971f;
}

.delete-btn {
    background-color: #d9534f;
    color: white;
    border: none;
    padding: 5px 8px;
    border-radius: 3px;
    cursor: pointer;
}

.delete-btn:hover {
    background-color: #b52b27;
}
```

app.js

```
document.addEventListener('DOMContentLoaded', loadStudents);
```

```
const studentForm = document.getElementById('studentForm');
const studentTableBody = document.getElementById('studentTableBody');
const formTitle = document.getElementById('formTitle');
const submitBtn = document.getElementById('submitBtn');
const cancelBtn = document.getElementById('cancelBtn');
const editIndexInput = document.getElementById('editIndex');

studentForm.addEventListener('submit', function(e) {
  e.preventDefault();

  const name = document.getElementById('studentName').value;
  const rollNo = document.getElementById('rollNo').value;
  const course = document.getElementById('course').value;
  const editIndex = editIndexInput.value;

  let students = getStoredStudents();

  if (editIndex === "") {
    // Add New Record
    const student = { name, rollNo, course };
    students.push(student);
  } else {
    // Update Existing Record
    students[editIndex] = { name, rollNo, course };
```

```
    }

    localStorage.setItem('students', JSON.stringify(students));
    refreshTable();
    resetForm();
});

function getStoredStudents() {
    return JSON.parse(localStorage.getItem('students')) || [];
}

function loadStudents() {
    const students = getStoredStudents();
    studentTableBody.innerHTML = '';
    students.forEach((student, index) => {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${student.name}</td>
            <td>${student.rollNo}</td>
            <td>${student.course}</td>
            <td>
                <div class="action-btns">
                    <button class="edit-btn"
onclick="editStudent(${index})">Edit</button>
                    <button class="delete-btn"
onclick="deleteStudent(${index})">Delete</button>
```

```
            </div>
          </td>
        `;
        studentTableBody.appendChild(row);
    });
}

function editStudent(index) {
    const students = getStoredStudents();
    const student = students[index];

    document.getElementById('studentName').value = student.name;
    document.getElementById('rollNo').value = student.rollNo;
    document.getElementById('course').value = student.course;
    editIndexInput.value = index;

    formTitle.textContent = "Edit Student Record";
    submitBtn.textContent = "Update Record";
    cancelBtn.style.display = "block";
}

function resetForm() {
    studentForm.reset();
    editIndexInput.value = "";
    formTitle.textContent = "Add New Student";
```

```
    submitBtn.textContent = "Add Record";
    cancelBtn.style.display = "none";
}

function deleteStudent(index) {
    let students = getStoredStudents();
    students.splice(index, 1);
    localStorage.setItem('students', JSON.stringify(students));
    refreshTable();
    resetForm();
}

function refreshTable() {
    studentTableBody.innerHTML = '';
    loadStudents();
}
```

sw.js

```
const CACHE_NAME = 'student-pwa-v1';
const ASSETS = [
    'index.html',
    'style.css',
    'app.js',
    'manifest.json'
```

```
];

self.addEventListener('install', (e) => {
  e.waitUntil(
    caches.open(CACHE_NAME).then((cache) => {
      return cache.addAll(ASSETS);
    })
  );
});

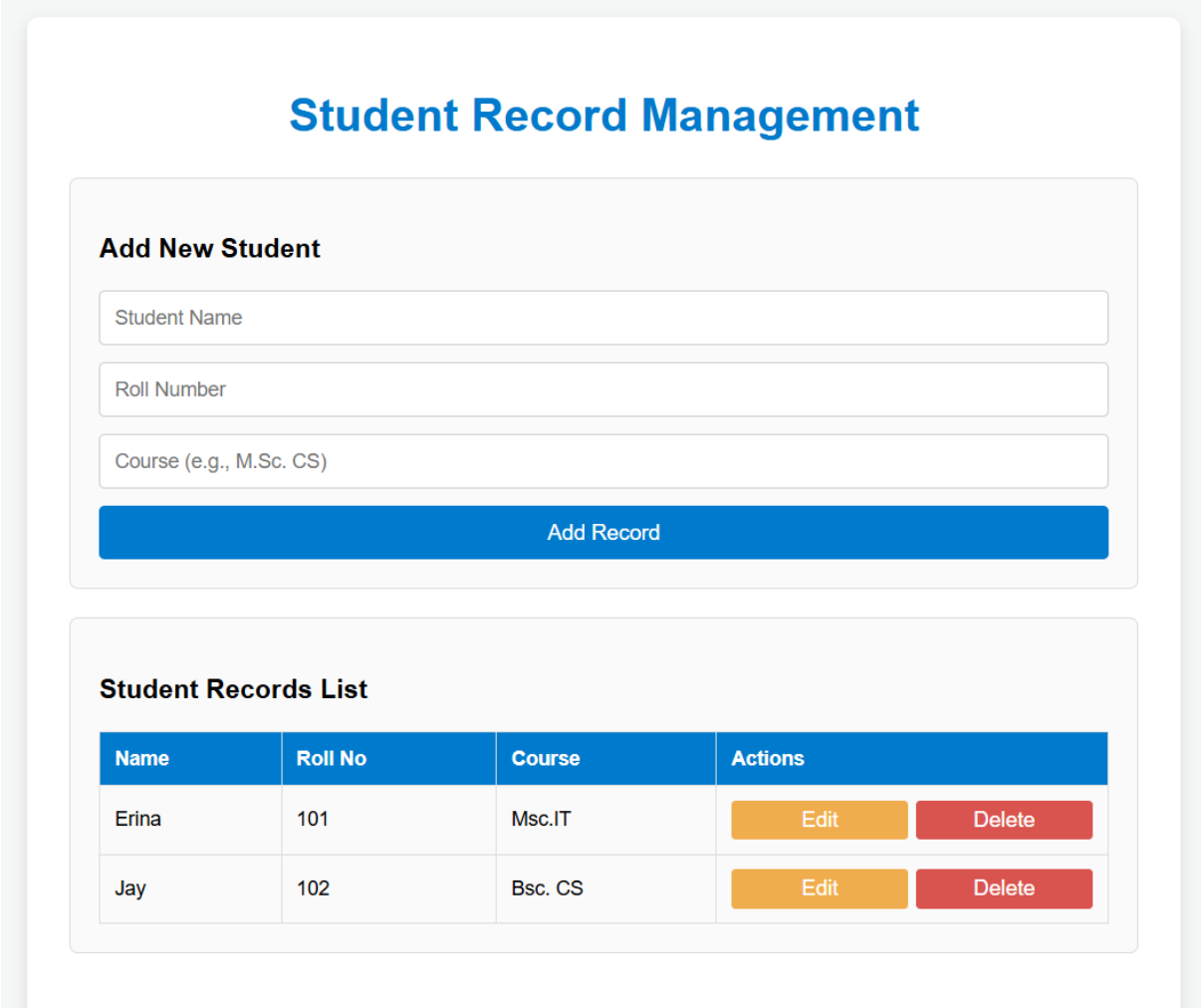
self.addEventListener('fetch', (e) => {
  e.respondWith(
    caches.match(e.request).then((response) => {
      return response || fetch(e.request);
    })
  );
});
```

manifest.json

```
{
  "name": "Student Record Management PWA",
  "short_name": "StudentPWA",
  "start_url": "index.html",
  "display": "standalone",
```

```
  "background_color": "#f4f7f6",
  "theme_color": "#007acc",
  "icons": []
}
```

Output:



Practical No: 8

a. **Aim: Create an HTML page that would be the starting point of the application. This HTML will contain a link to the file named manifest.json. This is an important file that would be created in the next step.**

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Record Portal - Home</title>

  <!-- Linking the Manifest File (Created in the next step) -->
  <link rel="manifest" href="manifest.json">

  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Welcome to Student Portal</h1>
    <p>This is the starting point of our Progressive Web Application (PWA).</p>
    <p>The application is linked with <code>manifest.json</code> to enable installation and offline capabilities.</p>
```

```
  </div>
</body>
</html>
```

manifest.json

```
{
  "name": "Student Record Management Portal",
  "short_name": "StudentPortal",
  "start_url": "index.html",
  "display": "standalone",
  "background_color": "#f4f7f6",
  "theme_color": "#007acc",
  "description": "A Progressive Web App for managing student records offline.",
  "icons": [
    {
      "src": "icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
```

```
  }
 ]
}
```

Output:



b. Create a simple Web API that returns Student data in JSON format.

Steps:

First type in terminal npm -v if its shows version then no need to install node js if it shows error download node js

<u>Steps to install node js</u>

Step 1: Download Node.js

Go to your browser and search for—or directly visit—nodejs.org.

You will see two versions there. You need to download the LTS (Long Term Support) version (this is the most stable version).

Step 2: Install It

Once downloaded, double-click the .msi file to open it.

In the installation wizard, simply click Next > Next > Next, and finally click Install. (Ensure the "Add to PATH" option is checked; it is selected by default).

Step 3: Restart VS Code (Most Important!)

After the installation is complete, close VS Code completely and reopen it so the terminal can detect the new path.

Now, reopen your project folder and run this command in the terminal to verify: npm -v

After downloading node js and return it shows error follow the below steps

Simply type this command into your terminal and press Enter:

PowerShell

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

When it asks for confirmation (such as [Y] Yes), type Y and press Enter.

Now start with the Main practicals

Step 1: Project Setup (in the terminal)

Run these commands in your project folder's terminal to install the Express package:

npm init -y

npm install express

Step 2: Server.js

Create a new file in your folder named server.js

```
const express = require('express');
const app = express();
const PORT = 3000;

// Sample Student Data in JSON format
const studentsData = [
{ id: 1, rollNo: "101", name: "Aarti", course: "M.Sc. Computer Science" },
{ id: 2, rollNo: "102", name: "Rahul Sharma", course: "BCA" },
{ id: 3, rollNo: "103", name: "Priya Singh", course: "B.Tech IT" }
];

// Define a GET endpoint for students API
app.get('/api/students', (req, res) => {
res.json({
success: true,
count: studentsData.length,
students: studentsData
});
```

```
});

//Start the server
app.listen(PORT, () => {
console.log(`Server is running at http://localhost:${PORT}`);
});
```

How to Run & Test:

1. **Start the server by typing this command in the terminal:**

   ```
   node server.js
   ```

2. **Open your browser and type this link in the address bar:**

   http://localhost:3000/api/students

Output:

